



Anatomy of a C Program

Memory Origination

Types of Memory

- Stack Memory
- Heap Memory

Stack

What is it?

Area of memory where we can add and remove data exhibiting a LIFO behavior.

Operations

Push: adding an element to a stack

Pop: remove element from a stack

Stacks are used for

- Local vars
- The call stack
- Places for return values

Heap

Larger than stack memory, things can be added or removed at any time.

Operations

- allocate memory: `malloc()`
- deallocate memory: `free()`

Things to keep in mind about the stack

- One stack per thread
- Stack Grows Down (towards heap that grows up)
- Stack memory may not always be protected (like in an OS or microcontroller)

Main Stack Pointer (MSP)

- In single threaded mode, only one stack, only use MSP
- Exclusively used in interrupts (!!)

Process Stack Pointer (PSP)

- Used in multithread mode as each thread has its own stack
- Holds address of current running thread
- Never used in interrupts

Heap Management

- Large area of memory used in dynamic allocation
- Application programs request memory (malloc()) and return it (free())
- The OS must manage the heap
 - What has been allocated and to what programs
- Every allocation must lead to deallocation
 - When users forget to free something --> **Memory Leak**

Memory Alignment

Definition:

Memory alignment is about how data is organized and accessed in a computer's memory.

- It's like organizing items in a storage unit to make them easier to find and retrieve.
- Depends on the processor, OS and compiler

Byte Access vs. Byte Addressing

- Memory can be byte-accessible without being byte-addressable

Imagine a library where you can read any book (byte-accessible), but you can only ask for books by shelf number, not individual book positions (not byte-addressable).

- In computer terms, this means you can read any byte of data, but you might not be able to directly access every single byte address.
- It's like being able to read any word in a book, but only being able to open the book to specific page numbers.

Trading Memory for Speed

Think of this like using pre-packaged meal kits instead of individual ingredients:

- **Standard Sizes:** The computer uses standard-sized "containers" for data, even if they're not completely full.
- **Faster Access:** By using these standard sizes, the computer knows exactly where to look for the data, making access quicker
- **Some Waste:** Like meal kits might include more of an ingredient than you need, this approach might use more memory than strictly necessary

Example:

Imagine storing numbers in boxes that can hold 4 items each:

[1][2][3][4]	[5][6][][]	[7][8][9][10]
Box 1	Box 2	Box 3

Even though Box 2 only has two numbers, it still uses a full box. This wastes some space but makes finding the start of each number group very fast and predictable.

This approach helps the computer work more efficiently, even if it uses a bit more memory. It's a trade-off between speed and memory usage, prioritizing quicker data access.

How do we ensure that data is always interpreted in the same way?

- Imagine you and your friend are playing with building blocks, but you're in different rooms.
- You describe a structure you built, but when your friend tries to recreate it, it looks different.

This is because different computer systems might interpret or store data structures differently.

Why do we care?

- OS developers often work with "arrays of bytes"
- They convert these bytes into different types of data
- If the data isn't lined up correctly (alignment), things won't work right

Bitfields (Packing ... Kinda)

Bitfields serve as a clever way to save space when storing information

- Imagine a box with 8 compartments that can each hold yes/no (0/1 bit) answers
- Instead of using 8 separate boxes for 8 yes/no questions you use one box with 8 compartments
- This is what bit fields do - they use individual bits in a larger chunk of memory to store multiple pieces of information
- They're organized from LSB to MSB, extras and padding

Global Variables

Global variables are stored in a "segment" - think of a segment as a dedicated shelf in a library where these books with this label (global variables) live